Step-by-step guide: deploying a 12-Factor App on Kubernetes using Minikube

Step 1: Prepare the application code and dependencies

The application is first written so that it aligns with the twelve factor principles, particularly regarding configuration and dependency management. Wiggins (2017) states that configuration values such as database credentials or API keys should be stored in environment variables rather than hard coded into the application, and that all dependencies should be explicitly declared so the application never relies on implicit system level packages. For this deployment, a simple Python Flask application was used, with dependencies declared in a requirements.txt file.

Step 2: Create a Dockerfile

A Dockerfile is written to containerise the application.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
ENV PORT=5000
EXPOSE 5000
CMD ["python", "app.py"]
```

The image is then built and tested locally.

```
docker build -t my-12factor-app:latest .
docker run -p 5000:5000 my-12factor-app:latest
```

Packaging the application this way reflects the factor concerning processes, which requires the application to run as a stateless, self contained process with no reliance on data persisted locally within the container.

Step 3: Start a local Kubernetes cluster with Minikube

A local cluster is created using Minikube, providing a lightweight environment in which to test deployment and scaling before moving to a production cluster.

```
minikube start
minikube status
```

Step 4: Load the image and define the deployment

The image is loaded into Minikube and a deployment manifest is created.

```
minikube image load my-12factor-app:latest
apiVersion: apps/v1
kind: Deployment
metadata:
```

```
  name: my-12factor-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: my-12factor-app
  template:
    metadata:
      labels:
        app: my-12factor-app
    spec:
      containers:
        - name: my-12factor-app
          image: my-12factor-app:latest
          ports:
            - containerPort: 5000
          envFrom:
            - configMapRef:
                name: app-config
```

Configuration values are supplied through a ConfigMap rather than being embedded in the image, reflecting the strict separation between configuration and code.

```
kubectl apply -f deployment.yaml
```

Step 5: Expose the application through a service

A Kubernetes service is defined to expose the deployment.

```
apiVersion: v1
kind: Service
metadata:
  name: my-12factor-app-service
spec:
  type: NodePort
  selector:
    app: my-12factor-app
  ports:
    - port: 5000
      targetPort: 5000
kubectl apply -f service.yaml
minikube service my-12factor-app-service
```

Step 6: Verify scaling and disposability

The deployment is scaled to confirm that additional pods start up quickly and terminate gracefully, reflecting the disposability factor.

```
kubectl scale deployment my-12factor-app --replicas=3
kubectl get pods
```

This process demonstrates how container orchestration platforms such as Kubernetes support the twelve factor methodology in practice. Hamzaoui et al.

(2024) note that container based architectures address many of the portability and scalability challenges associated with traditional deployment models, while introducing their own considerations around orchestration complexity and resource management, both of which are visible in the steps carried out above. The underlying resource management principles behind this kind of elastic scaling share conceptual ground with the wider work on distributed computing environments discussed by Harshit et al. (2016), even though their toolkit itself is applied to edge and fog computing rather than container orchestration directly. Using Minikube for local testing allows these behaviours to be observed and validated before an application is deployed to a larger, production grade cluster.

References

Hamzaoui, I. et al. (2024) 'A topical review on container based cloud revolution: multi directional challenges, and future trends', *SN Computer Science*, 5(1), pp. 1 to 20.

Harshit, G. et al. (2016) *iFogSim: a toolkit for modeling and simulation of resource management techniques in Internet of Things, edge and fog computing environments*.

Wiggins, A. (2017) *The twelve factor app*. Available at: https://12factor.net (Accessed: 22 August 2026).